



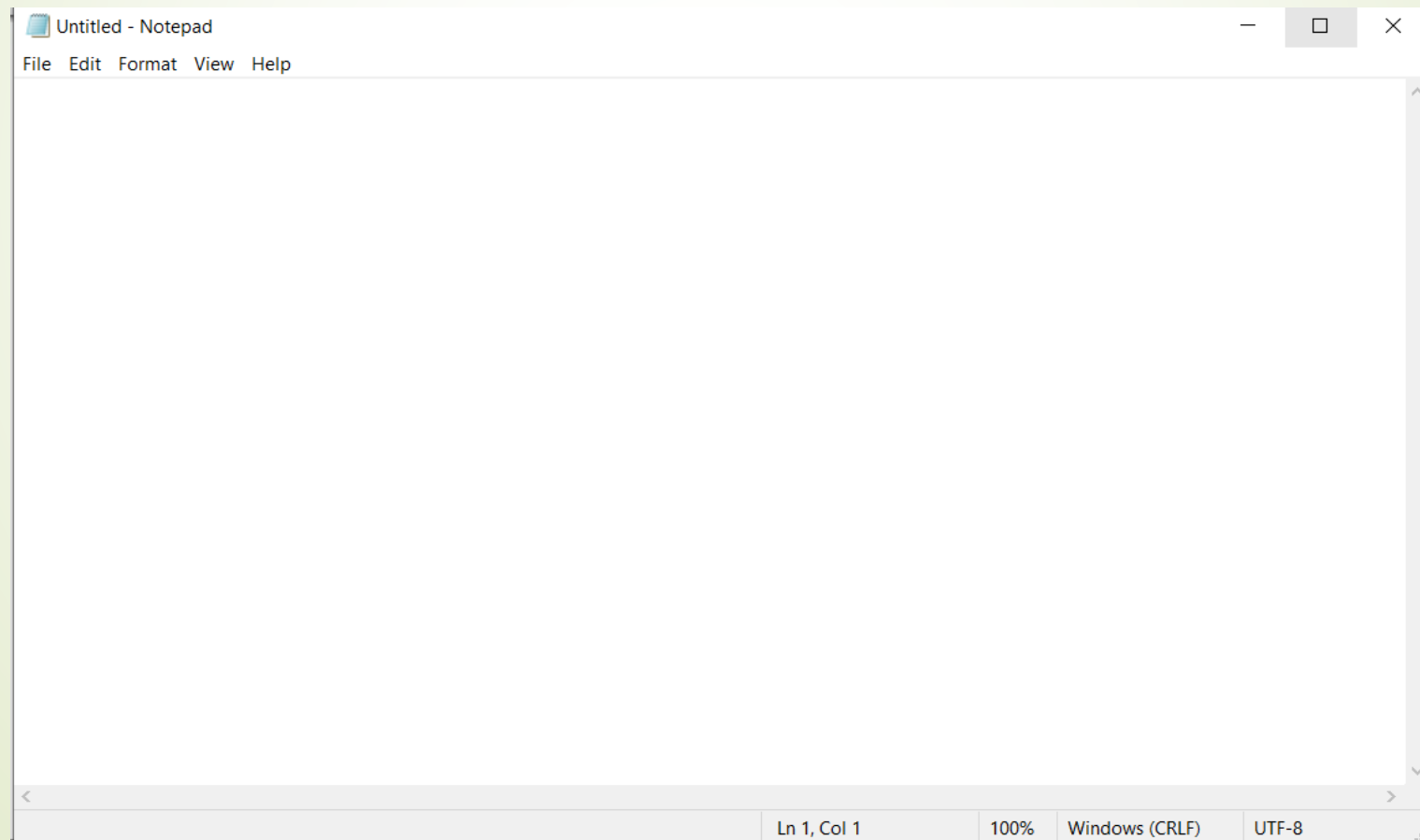
Lecture_2

Windows Programming_II

Dr. Sara Mohamed

Single-document interface (SDI) applications.

- These applications allow a user to work with only one window at a time.



Multiple-document interface (MDI) applications

- Several large Windows applications, such as Microsoft Excel and Visual Studio .NET, allow users to work with several open windows at the same time for example **Photoshop** application.
- The main application window of an MDI application acts as the **parent window**, which can open several **child** window.
- **Multiple-document interface (MDI)** applications enable you to display multiple documents at the same time, with each document displayed in its own window. MDI applications often have a **Window menu item with submenus** for switching between windows or documents.



➤ The main application window of an MDI application acts as the **parent** window, which can open several **child** windows. You need to know the following main points about an MDI application:

- Child windows are restricted to their parent window (that is, you cannot move a child window outside its parent window).
- The parent window can open several types of child windows. As an example of an MDI application, Visual Studio .NET allows you to work with several types of document windows at the same time.
- The child windows can be opened, closed, maximized, or minimized independently of each other, but when the parent window is closed, the child windows are automatically closed.

- The MDI frame should always have a **menu**; one of the menus that a user always expects to see in an MDI application is the Window menu, which allows the user to manipulate various windows that are open in an MDI container form.



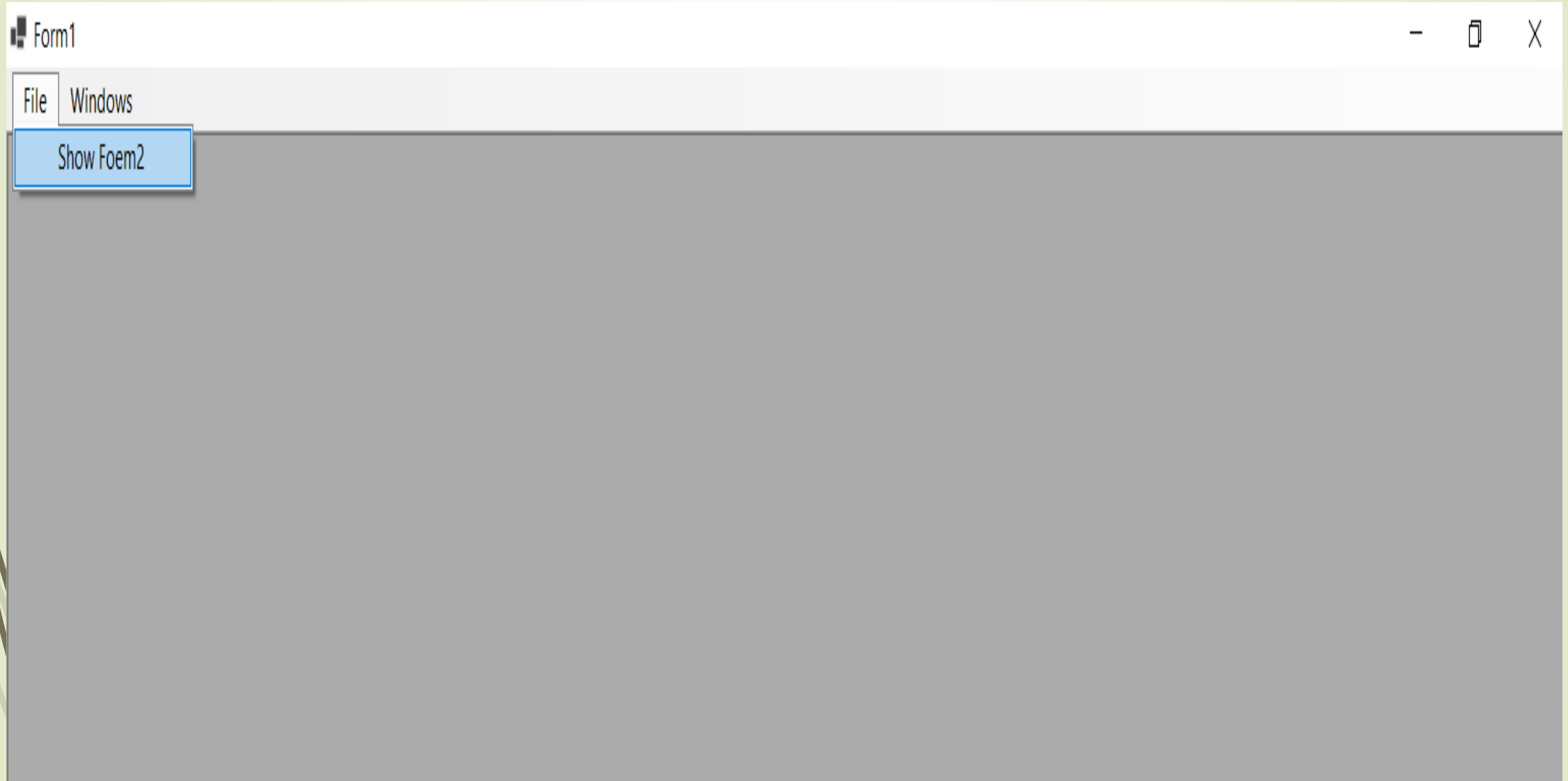
IsMdiContainer property to true

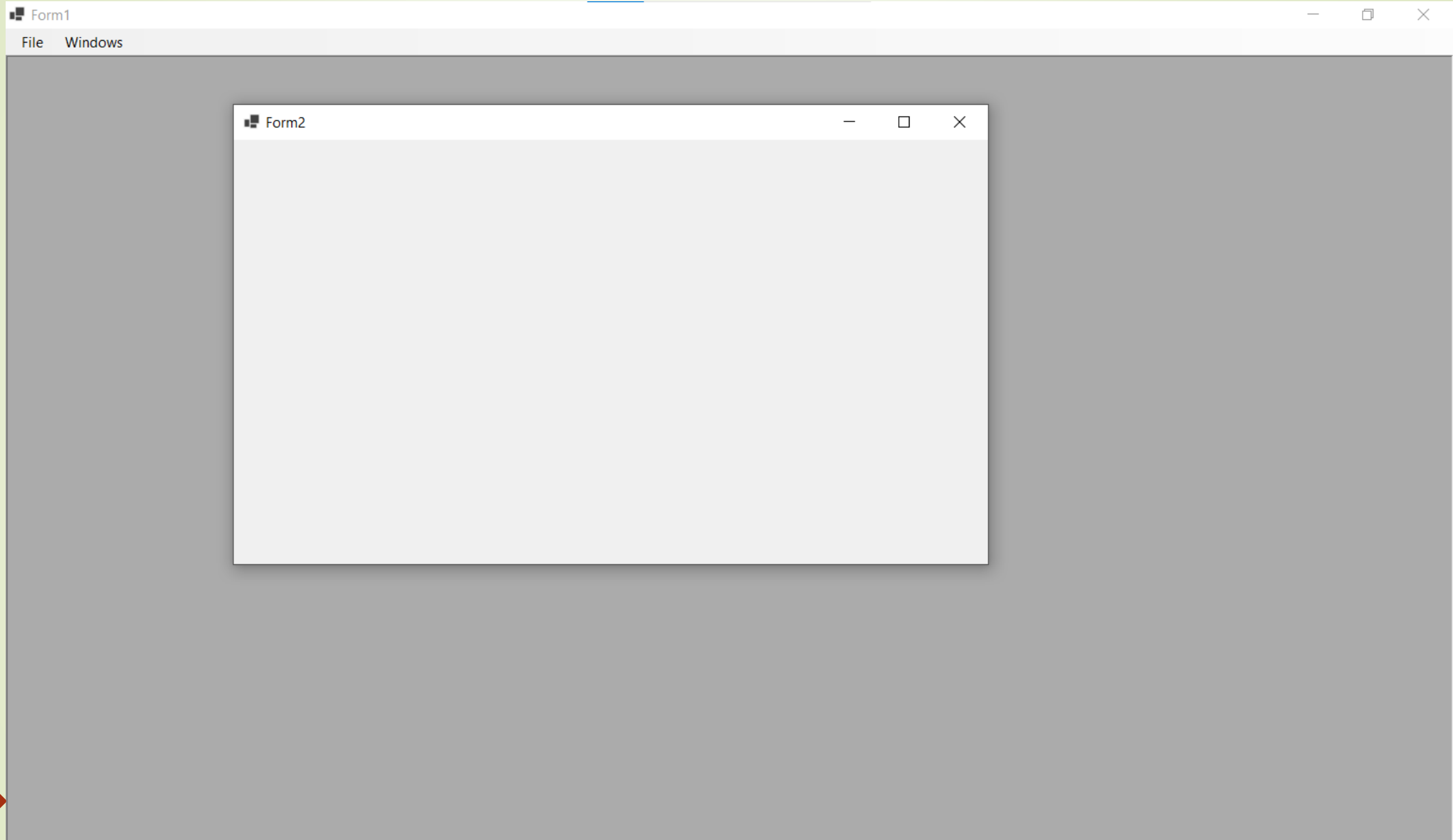
- The Windows Forms in an MDI application are also created by using the standard `System.Windows.Forms.Form` class. To create an **MDI parent form**, you create a regular Windows form and change its **IsMdiContainer** property to true.

IMPORTANT MEMBERS OF THE Form CLASS THAT ARE RELATED TO MDI APPLICATIONS

<i>Member</i>	<i>Type</i>	<i>Description</i>
ActiveMdiChild	Property	Identifies the currently active MDI child window
IsMdiContainer	Property	Indicates whether the form is a container for MDI child forms
MdiChildActivate	Event	Occurs when an MDI child form is activated or closed within an MDI application
MdiChildren	Property	Specifies an array of forms that represent the MDI child form of the form
MdiParent	Property	Specifies the MDI parent form for the current form
LayoutMdi()	Method	Arranges the MDI child forms within an MDI parent form

Example_1





1. Add **MenuStrip** to Form1 window, then add item and make the text property it to File .

2. make the isMdiContainer property for the form to True

3. make the WindowState to Maximize

The screenshot shows the Visual Studio IDE with three tabs: Form2.cs [Design], Form1.cs, and Form1.cs [Design]. A yellow notification bar at the top states: "Scaling on your main display is set to 125%. Restart Visual Studio with 100% scaling. Learn more...". The main design area displays a window titled "Form1" with a menu bar containing "File" and "Windows". The "File" menu is expanded, showing a single item. The Properties window on the right is set to "Form1" (System.Windows.Forms.Form). The "MainMenuStrip" property is set to "menuStrip1". The "IsMdiContainer" property is set to "True". The "WindowState" property is set to "Maximized". The "Size" property is set to "919, 633". The "Text" property is set to "Form1".

Property	Value
Enabled	True
Font	Segoe UI, 9pt
ForeColor	ControlText
FormBorderStyle	Sizable
HelpButton	False
Icon	(Icon)
ImeMode	NoControl
IsMdiContainer	True
KeyPreview	False
Language	(Default)
Localizable	False
Location	0, 0
Locked	False
MainMenuStrip	menuStrip1
MaximizeBox	True
MaximumSize	0, 0
MdiChildrenMinimizedA	True
MinimizeBox	True
MinimumSize	0, 0
Opacity	100%
Padding	0, 0, 0, 0
RightToLeft	No
RightToLeftLayout	False
ShowIcon	True
ShowInTaskbar	True
Size	919, 633
SizeGripStyle	Auto
StartPosition	WindowsDefaultLoca
Tag	
Text	Form1
TopMost	False

 2 Set from Properties Window (Visual Studio)

1. Select the Form
2. Go to **Properties**
3. Find **WindowState**
4. Choose **Maximized**

 No code needed 

 3 WPF (C#)

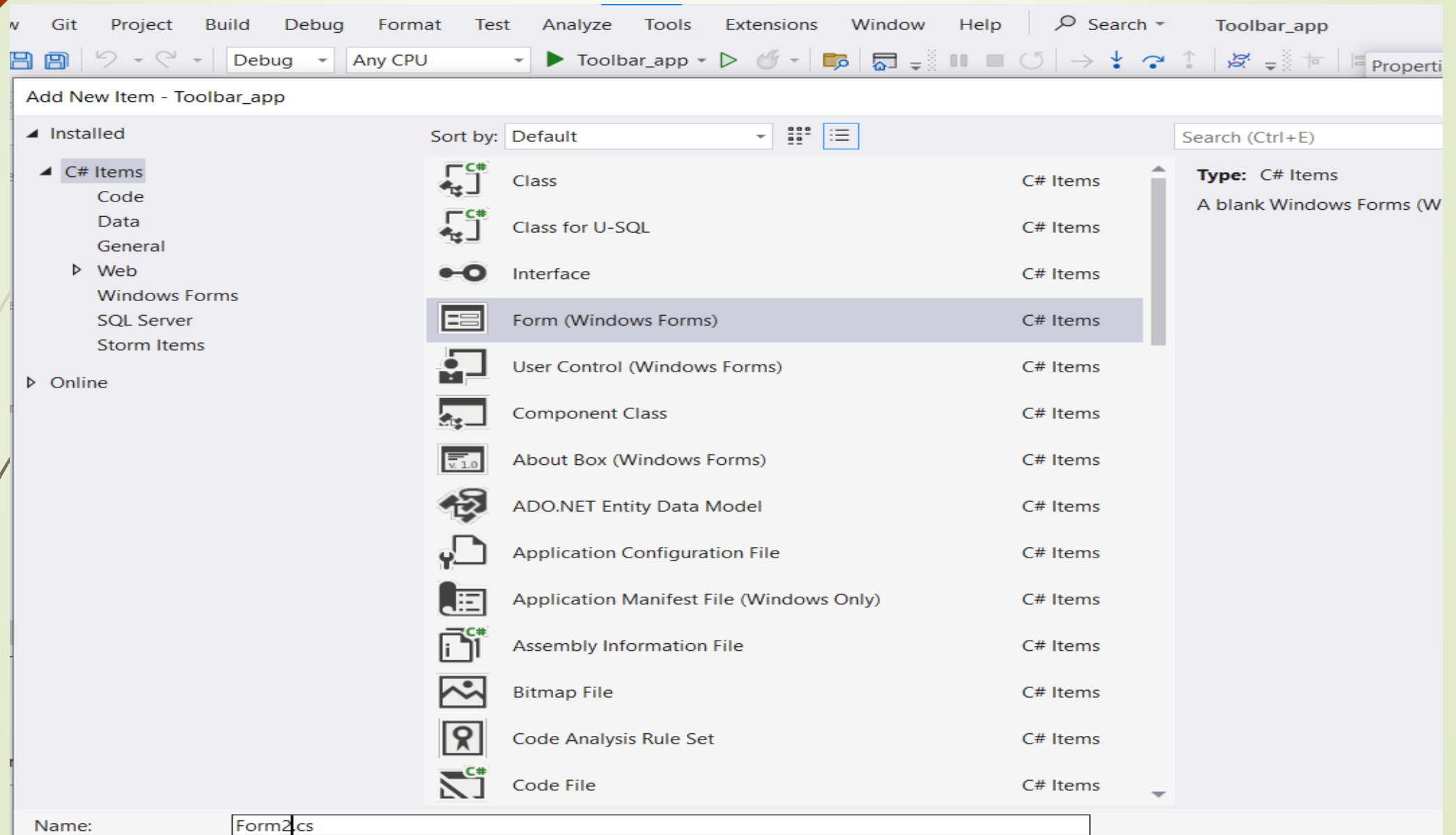
 C#

```
this.WindowState = WindowState.Maximized;
```

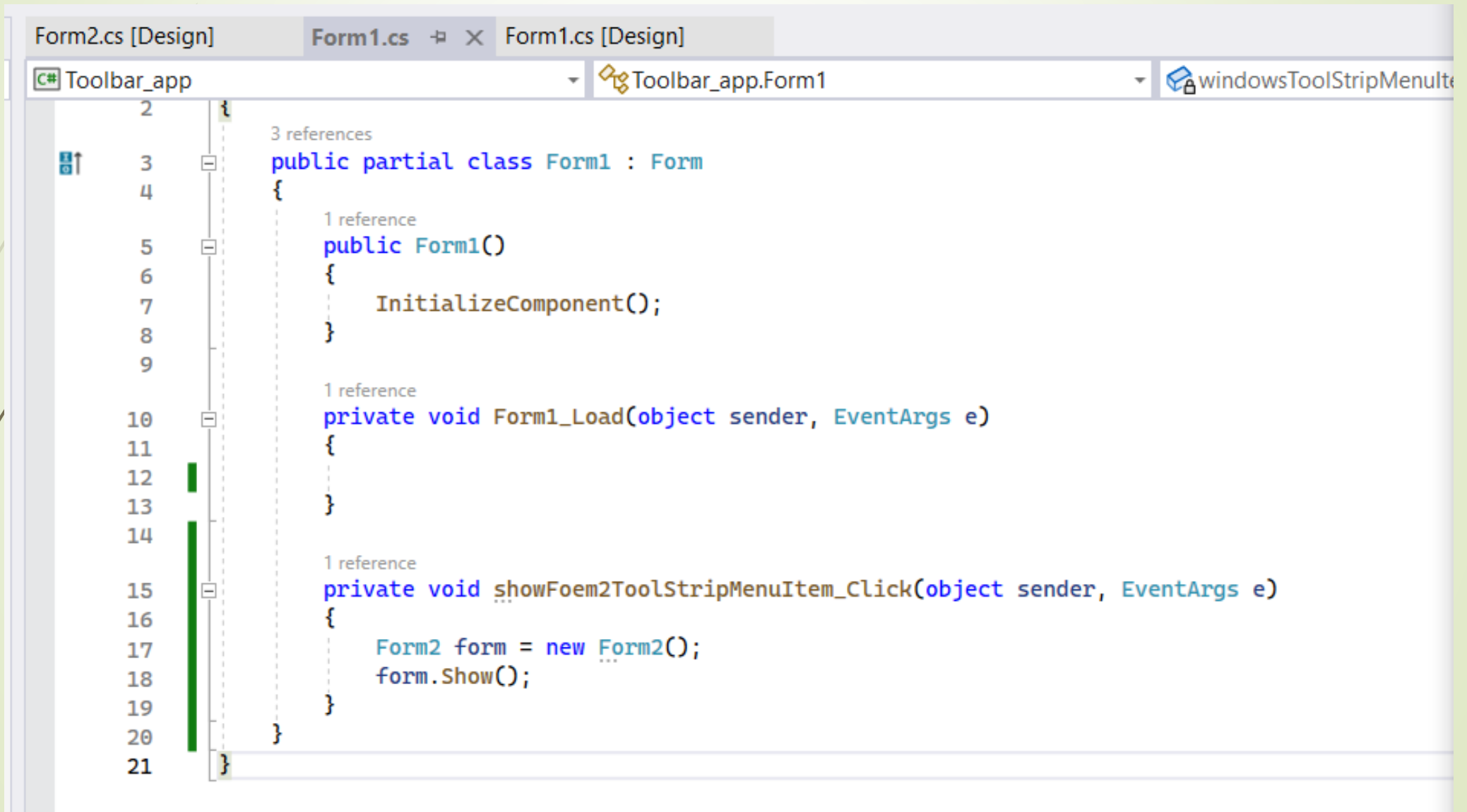
 Result

The window will open **maximized** automatically   

Add new windows form from Project tab



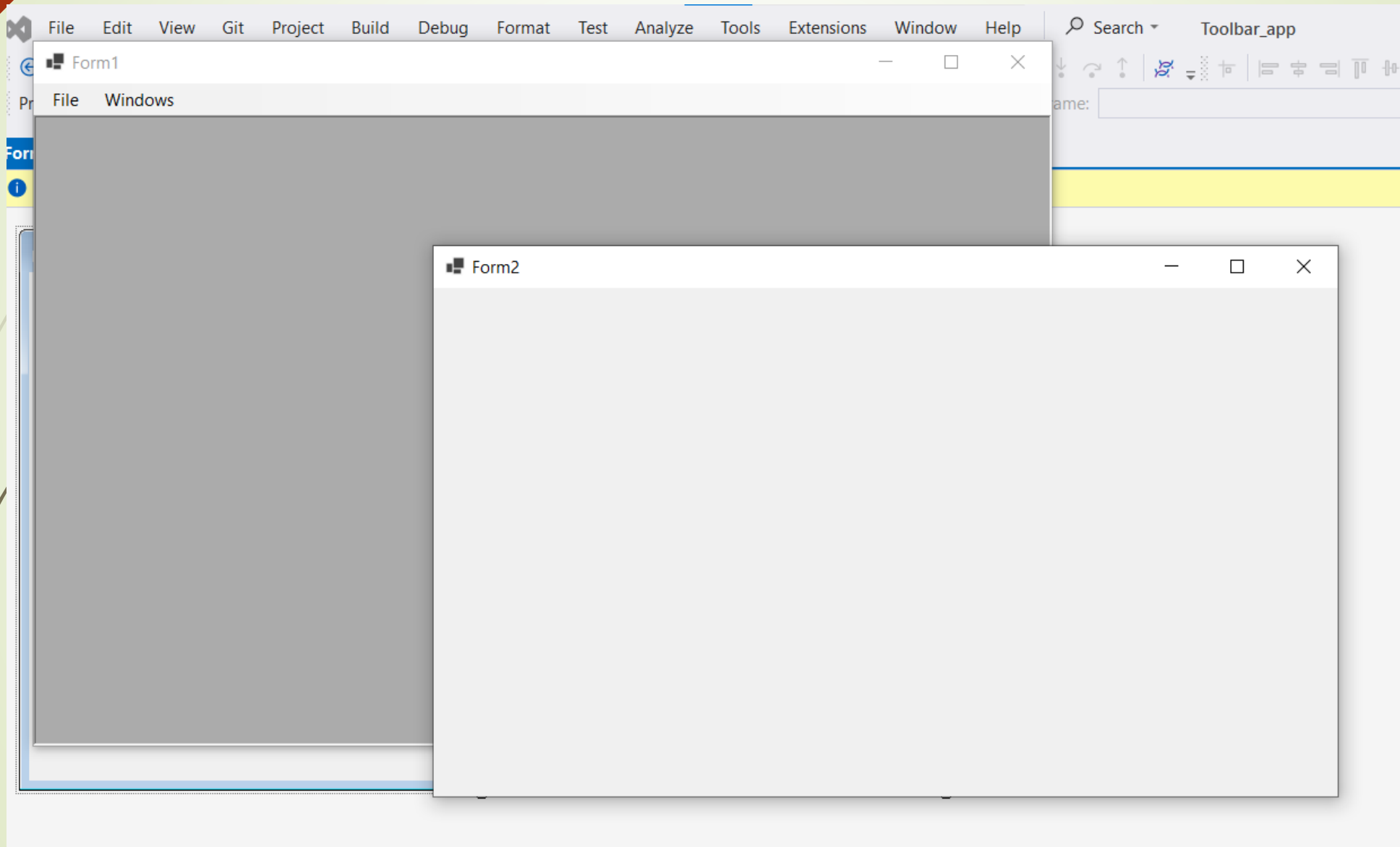
Open the click event for the **ToolStripMenuItem** then type the following code:



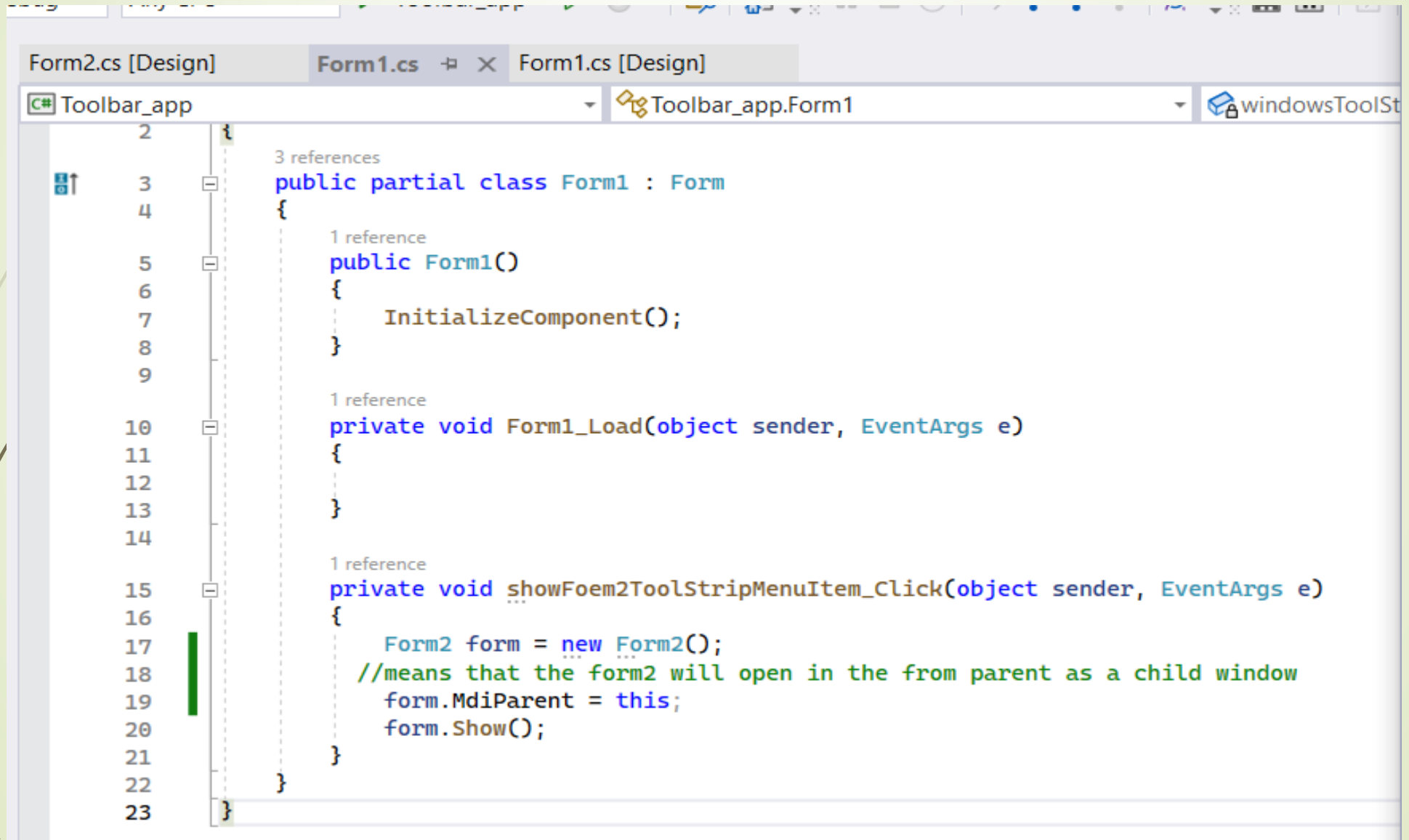
```
Form2.cs [Design] | Form1.cs | Form1.cs [Design]
C# Toolbar_app | Toolbar_app.Form1 | windowsToolStripMenuIt

2 | }
3 | 3 references
4 | public partial class Form1 : Form
5 | {
6 |     1 reference
7 |     public Form1()
8 |     {
9 |         InitializeComponent();
10 |     }
11 |
12 |     1 reference
13 |     private void Form1_Load(object sender, EventArgs e)
14 |     {
15 |     }
16 |
17 |     1 reference
18 |     private void showFoem2ToolStripMenuItem_Click(object sender, EventArgs e)
19 |     {
20 |         Form2 form = new Form2();
21 |         form.Show();
22 |     }
23 | }
```

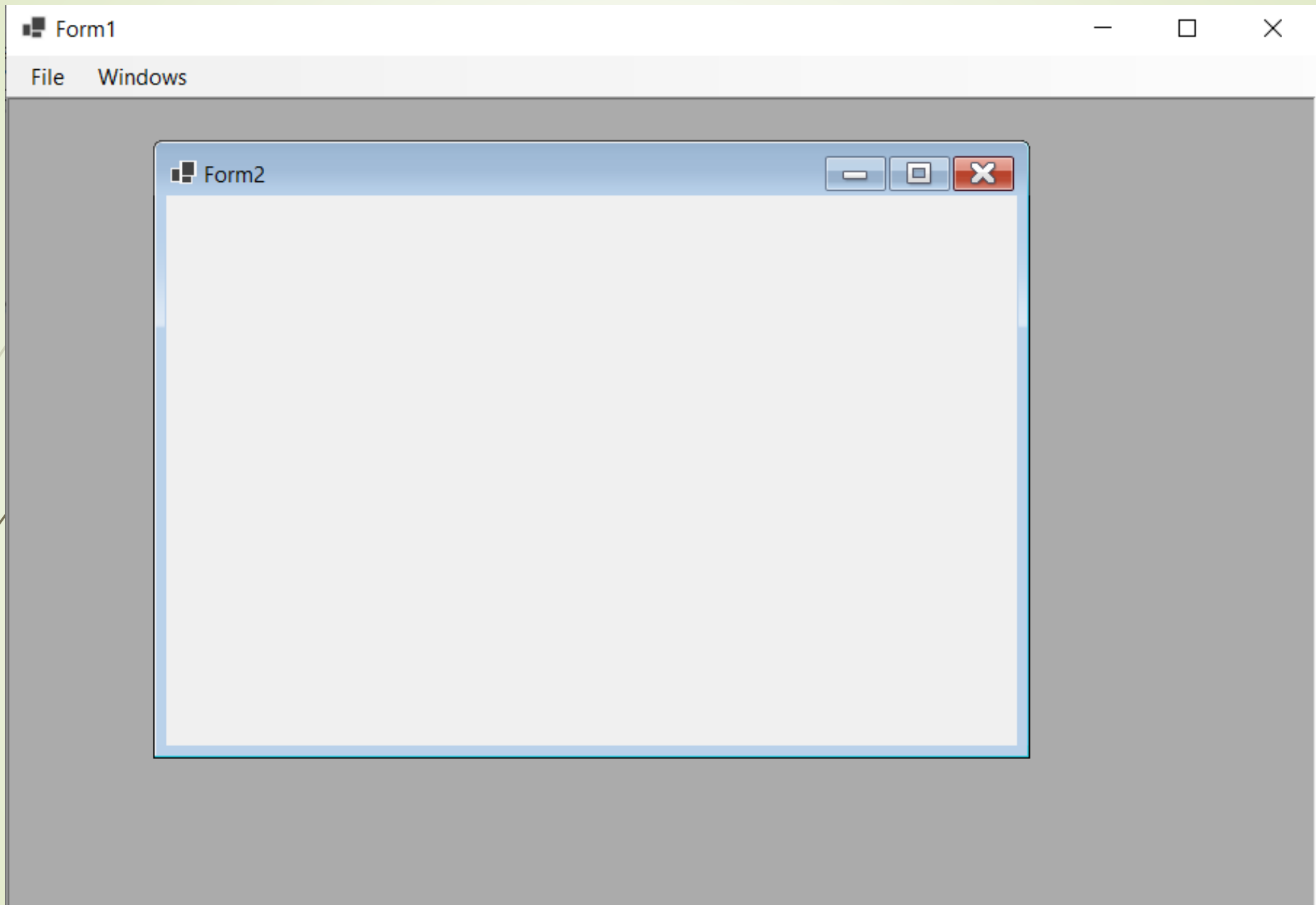
Output



MdiParent Property Specifies the MDI parent form for the current form



```
2 {
3     3 references
4     public partial class Form1 : Form
5     {
6         1 reference
7         public Form1()
8         {
9             InitializeComponent();
10        }
11
12        1 reference
13        private void Form1_Load(object sender, EventArgs e)
14        {
15        }
16
17        1 reference
18        private void showFoem2ToolStripMenuItem_Click(object sender, EventArgs e)
19        {
20            Form2 form = new Form2();
21            //means that the form2 will open in the from parent as a child window
22            form.MdiParent = this;
23            form.Show();
24        }
25    }
26 }
```

Example2

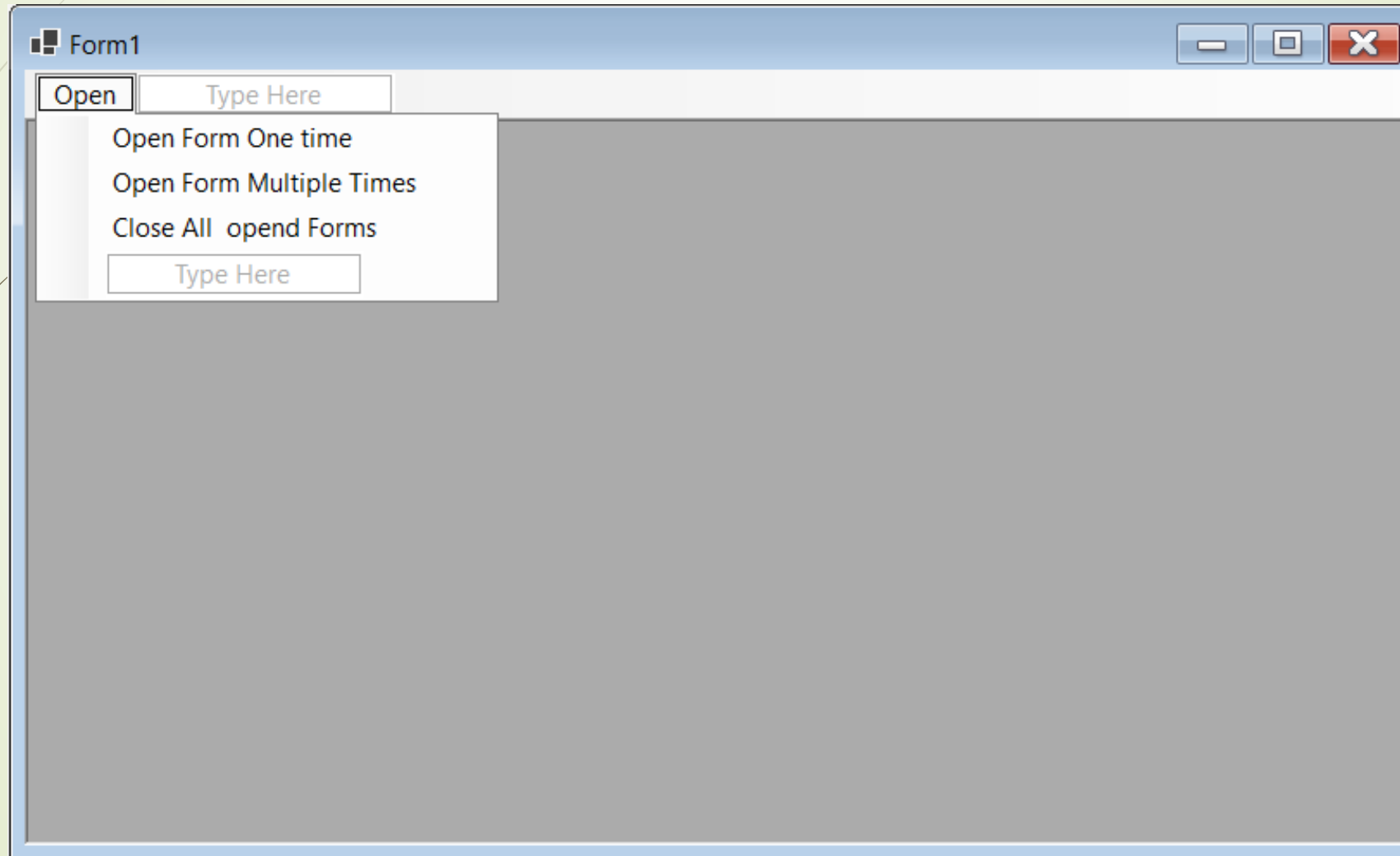
- Creating a MDI Parent ,(Create normal form & convert into MDI parent),
- Creating a MDI Child, (all other forms calling to MDI parent)
- Insert **Menu strip** in MDI Container and Insert some menu items,
- Open child form by clicking menu items as the following :
 - ☐ Form2 one time,
 - ☐ Form3 multiple times,
- Close All Opened Child forms by clicking menu item.
- Active form will not close.

The screenshot shows the Visual Studio IDE with a solution named 'multiple_form'. The 'Form1.cs [Design]' tab is active. A yellow notification bar at the top states: 'Scaling on your main display is set to 125%. Restart Visual Studio with 100% scaling. Learn more...'. The 'Form1' window is open, displaying a 'menuStrip1' control. The 'Properties' window on the right shows the following properties for 'Form1' (System.Windows.Forms.Form):

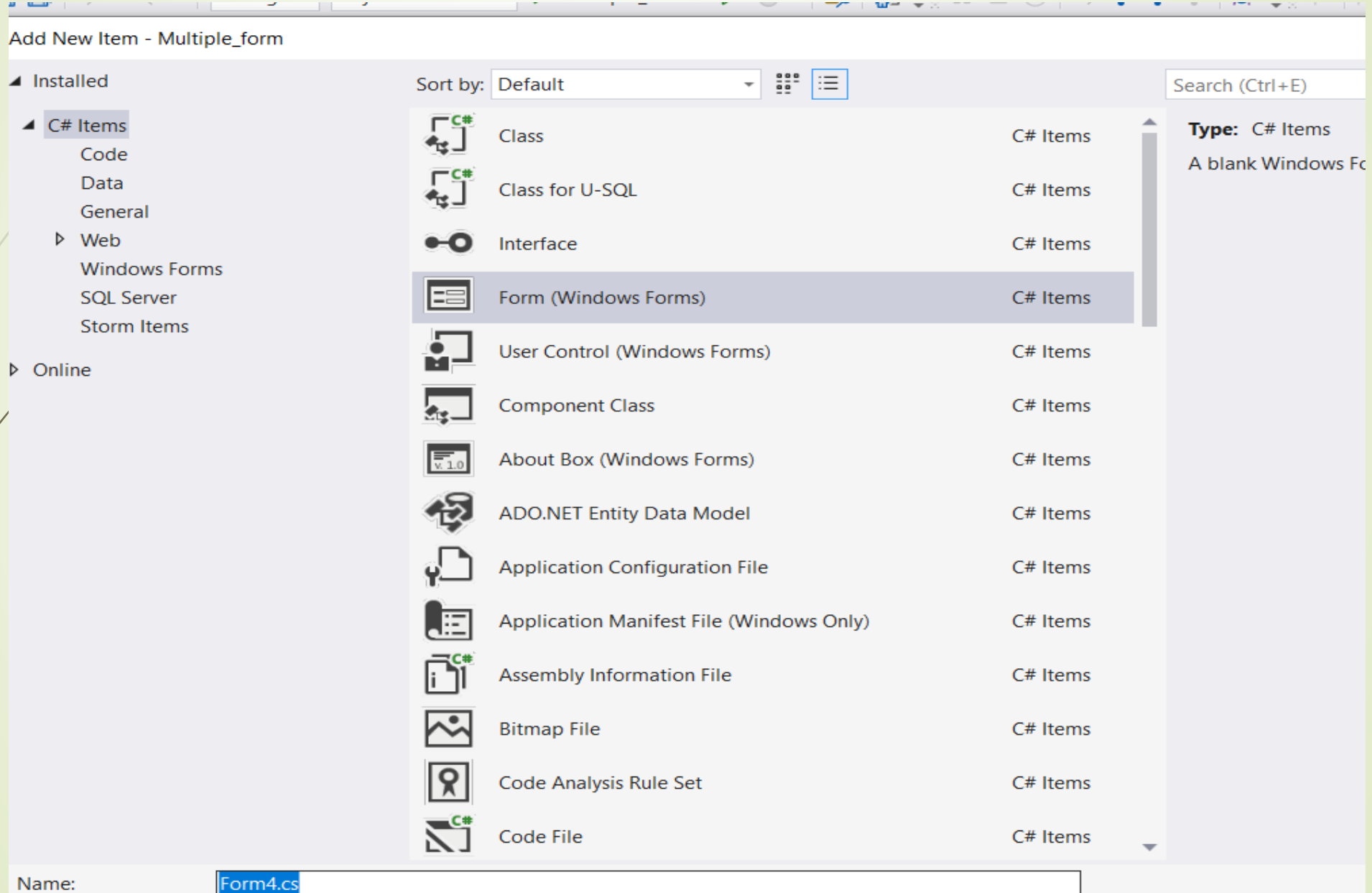
Property	Value
HelpButton	False
Icon	(Icon)
ImeMode	NoControl
IsMdiContainer	True
KeyPreview	False
Language	(Default)
Localizable	False
Location	0, 0
Locked	False
MainMenuStrip	menuStrip1
MaximizeBox	True
MaximumSize	0, 0
MdiChildrenMinimizedA	True
MinimizeBox	True
MinimumSize	0, 0
Opacity	100%
Padding	0, 0, 0, 0
RightToLeft	No
RightToLeftLayout	False
ShowIcon	True
ShowInTaskbar	True
Size	818, 497
SizeGripStyle	Auto
StartPosition	WindowsDefaultLoca
Tag	
Text	Form1
TopMost	False
TransparencyKey	
UseWaitCursor	False
WindowState	Maximized

Change the form1 Properties as the following

Add MenuStrips and add items as the following



Add two new windows forms(Form2 and Form3)



Open the click event for the openFormOne item then add this code

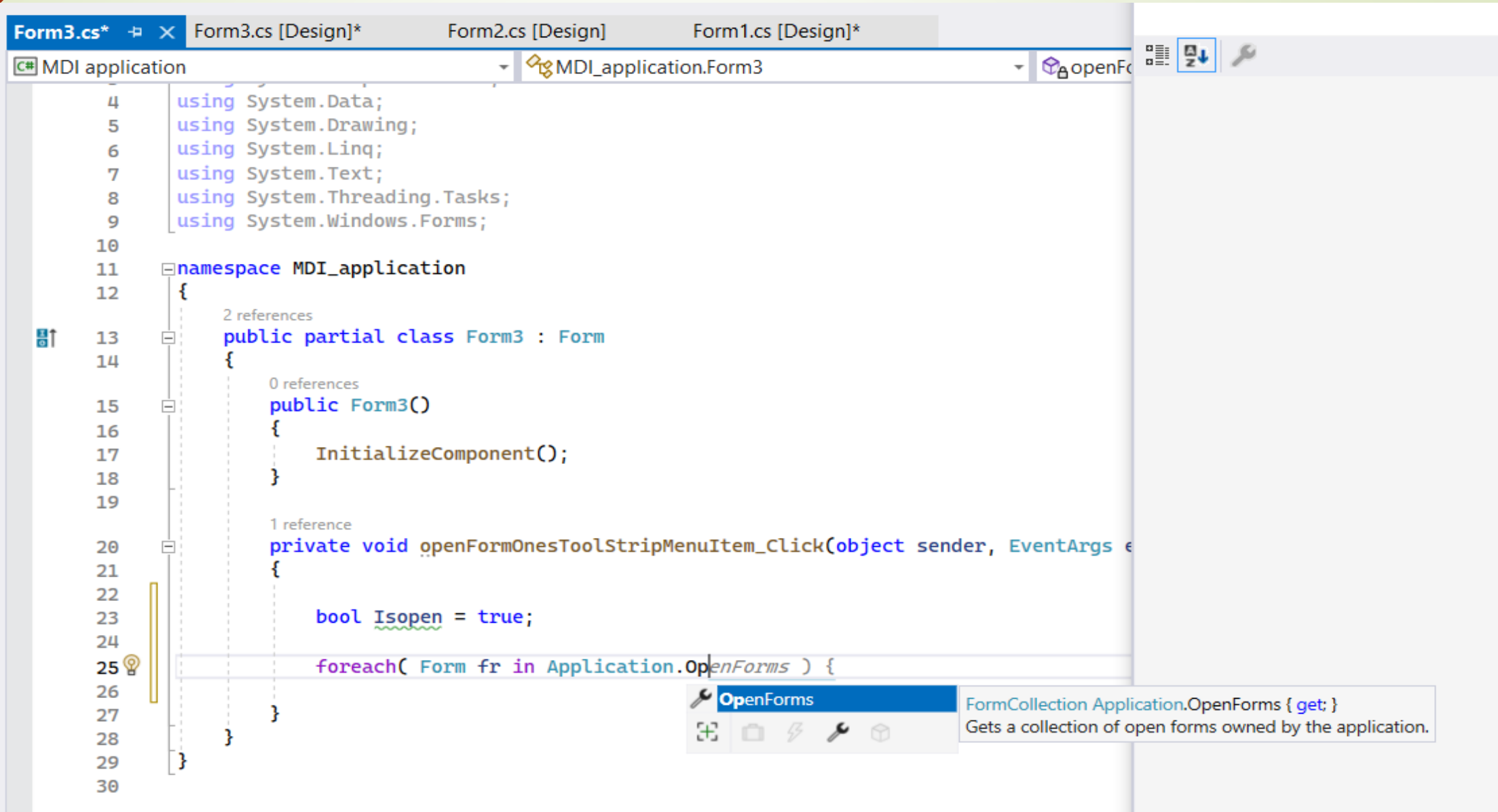
```
Form1.cs*  Form3.cs [Design]  Form2.cs [Design]  Form1.cs [Design]*
C# Multiple_form  Multiple_form.Form1  openFormOneTimeTool
11
12 // this form will open only one time
13 1 reference
14 private void openFormOneTimeToolStripMenuItem_Click(object sender, EventArgs e)
15 {
16     bool Isopen = false;
17
18     foreach (Form fr in Application.OpenForms)
19     {
20         if (fr.Text == "Form2")
21         {
22             Isopen = true;
23             // set the focus to the control
24             fr.Focus();
25             break;
26         }
27     }
28
29     if (Isopen == false)
30     {
31         Form2 form = new Form2();
32         form.MdiParent = this;
33         form.Show();
34     }
35 }
36
37
38
39
40
41
```

Collection of opened forms in the application

If Form2 window is opened then get the focus to it only and break

Open the form2 window in the parent form (Form1)

Application.OpenForms



```
4 using System.Data;
5 using System.Drawing;
6 using System.Linq;
7 using System.Text;
8 using System.Threading.Tasks;
9 using System.Windows.Forms;
10
11 namespace MDI_application
12 {
13     2 references
14     public partial class Form3 : Form
15     {
16         0 references
17         public Form3()
18         {
19             InitializeComponent();
20
21         1 reference
22         private void openFormOnesToolStripMenuItem_Click(object sender, EventArgs e)
23         {
24             bool Isopen = true;
25             foreach( Form fr in Application.OpenForms ) {
26
27             }
28         }
29     }
30 }
```

OpenForms
FormCollection Application.OpenForms { get; }
Gets a collection of open forms owned by the application.

Open the click event for the openFormMultipleTimes item then add this code

```
}
```

1 reference

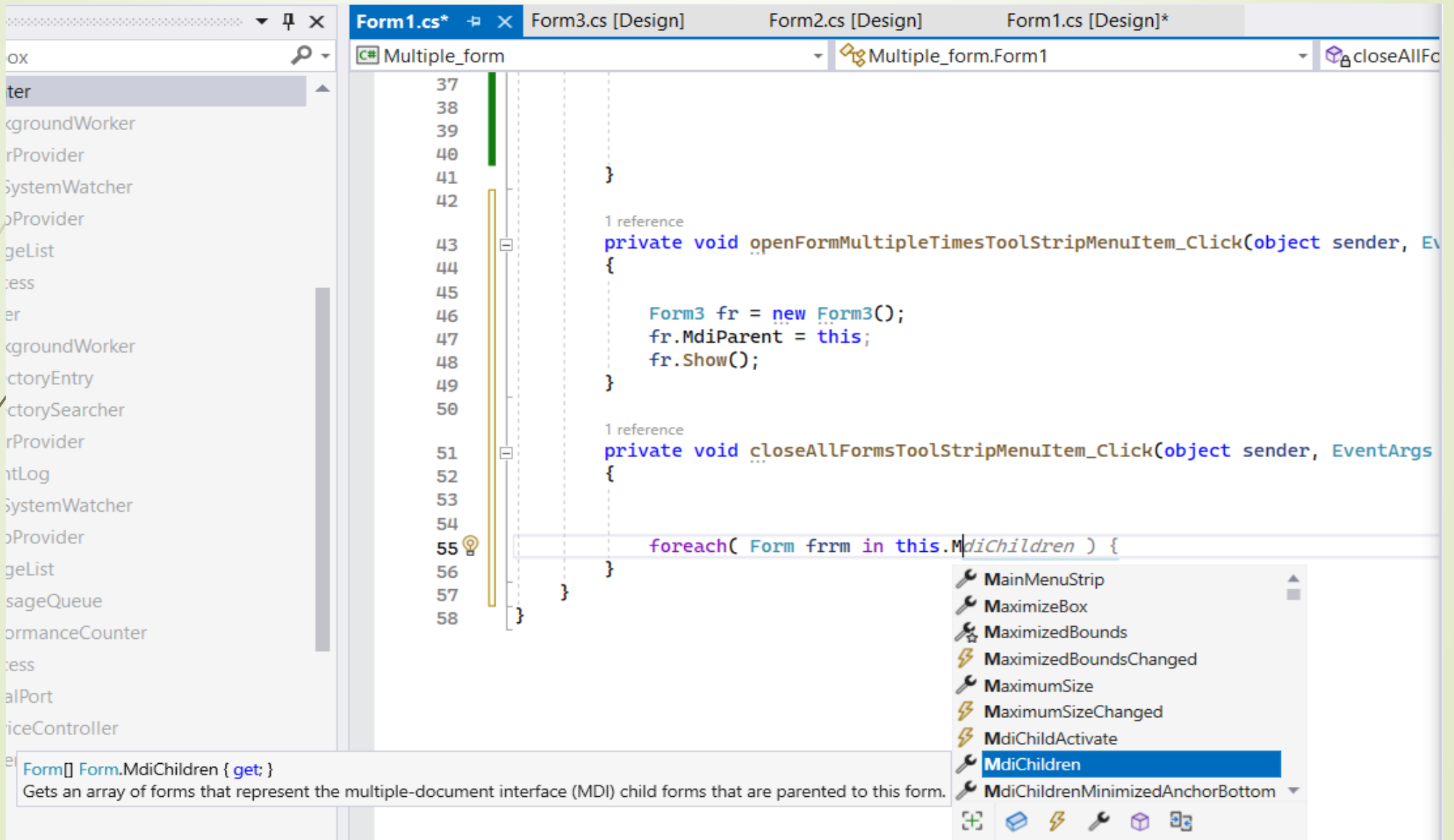
```
private void openFormMultipleTimesToolStripMenuItem_Click(object sender, EventArgs e)
{
    Form3 fr = new Form3();
    fr.MdiParent = this;
    fr.Show();
}
```

1 reference

Open the click event for the closeAllForms item
then add this code

```
1 reference
private void closeAllFormsToolStripMenuItem_Click(object sender, EventArgs e)
{
    foreach( Form frm in this.MdiChildren)
    {
        if(frm.Focus() == true)
        {
            frm.Visible = false;
            frm.Close();
            // frm.Dispose();
        }
    }
}
```

MdiChildren



The screenshot displays a Visual Studio IDE with the following components:

- File Explorer:** Located on the left, showing a project structure with folders like 'obj' and 'bin', and files like 'Form1.cs', 'Form2.cs', 'Form3.cs', and 'Multiple_form.cs'.
- Code Editor:** The main area showing the C# code for 'Form1.cs'. The code includes two event handlers for a menu strip:

```
37  
38  
39  
40  
41  
42  
43  
44  
45  
46  
47  
48  
49  
50  
51  
52  
53  
54  
55  
56  
57  
58
```

```
1 reference  
private void openFormMultipleTimesToolStripMenuItem_Click(object sender, EventArgs e)  
{  
    Form3 fr = new Form3();  
    fr.MdiParent = this;  
    fr.Show();  
}  
  
1 reference  
private void closeAllFormsToolStripMenuItem_Click(object sender, EventArgs e)  
{  
    foreach (Form frm in this.MdiChildren)  
    {  
        frm.Close();  
    }  
}
```
- Properties Window:** Located on the right, showing the 'MdiChildren' property of the 'Form1' object. The property is highlighted in blue.
- Tooltip:** A tooltip is visible at the bottom of the screen, providing information about the 'MdiChildren' property:

```
Form[] Form.MdiChildren { get; }  
Gets an array of forms that represent the multiple-document interface (MDI) child forms that are parented to this form.
```

Control.Focus Method

- The **Focus** method returns true if the control successfully received input focus.
- **Selectable** controls that can receive focus and show visual cues (like a blinking cursor, highlight, or selection border) when the user interacts with them.
- The control can have the input focus while not displaying any visual cues of having the focus. This behaviour is primarily observed by the **no selectable controls** listed below, or any controls derived from them.

Explanation

1. Focus Method:

- In GUI programming (like Windows Forms), many controls (buttons, textboxes, etc.) can receive "input focus."
- **Input focus** means that the control is ready to receive keyboard input.

2. Return Value (`true` or `false`):

- If you call the `Focus()` method on a control:
 - It returns `true` → The control **successfully received focus**.
 - It returns `false` → The control **did not receive focus**.

3. Focus Without Visual Cues:

- Normally, when a control has focus, you might see a blinking cursor or a highlighted border.
- But **some controls** can have focus **without showing any visual change**.
- Example: a `Label` or `Panel` usually doesn't highlight when focused.

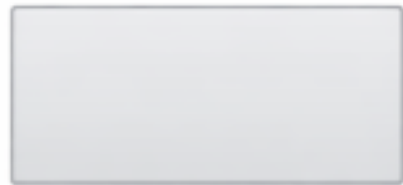
4. No Selectable Controls:

- The controls that don't show visual cues when focused are called "**non-selectable controls**".
- Any controls derived from these (custom controls based on them) will behave the same way.

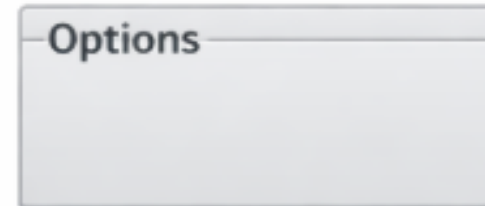
No Selectable Controls

Sample Text

Label



Panel



GroupBox

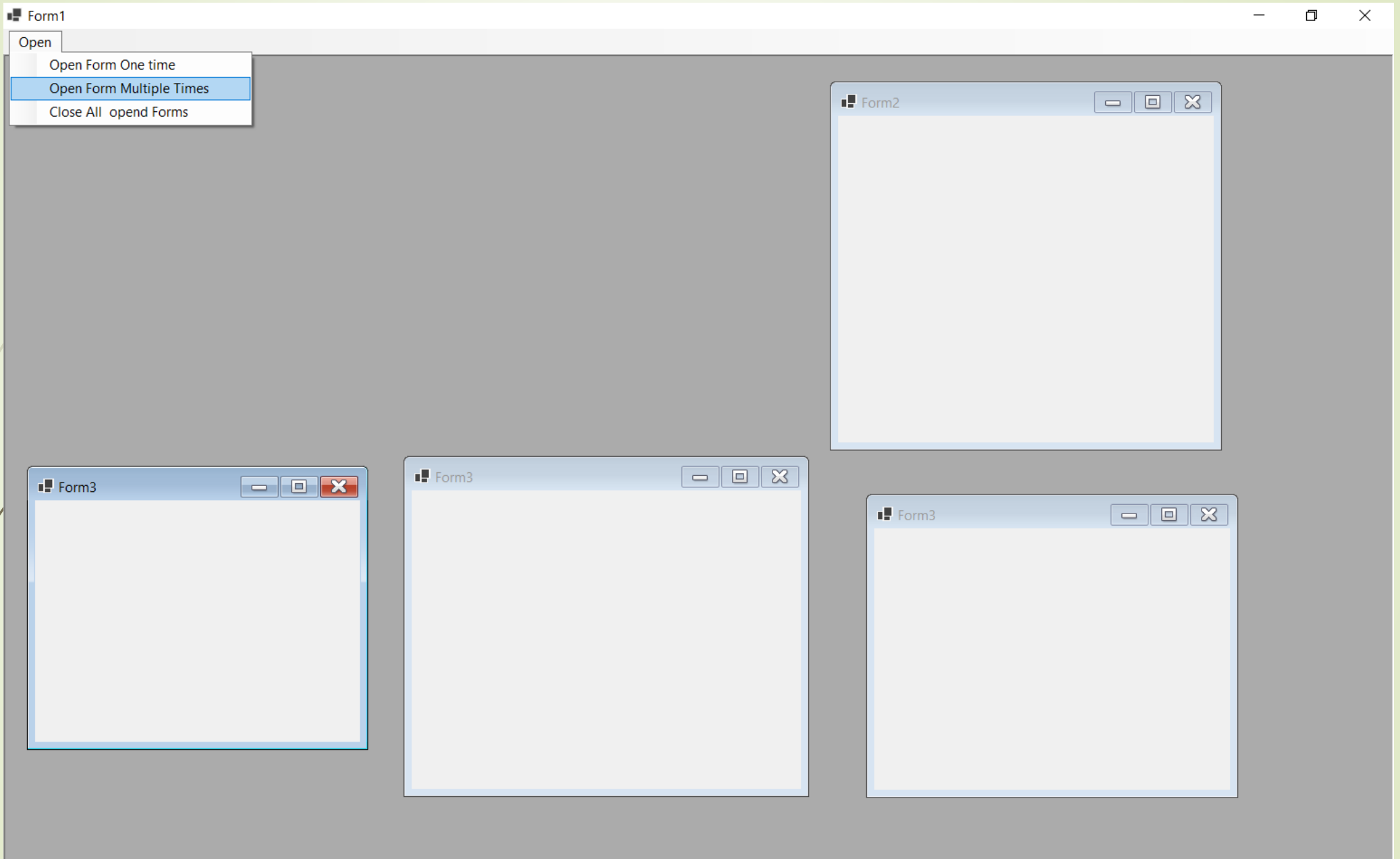


PictureBox

These controls cannot be selected by the user.

Selectable Controls

Control Type	Visual Cue / Behavior
TextBox	Shows a blinking cursor when focused
Button	Highlights or shows a border when clicked or focused
CheckBox	Shows a check mark when selected
RadioButton	Highlights the selected option
ComboBox / ListBox	Highlights the selected item
RichTextBox	Shows blinking cursor and highlights selected text



Printing tools

▷ Containers

▷ Menus & Toolbars

▷ Components

▲ Printing

 Pointer

 PageSetupDialog

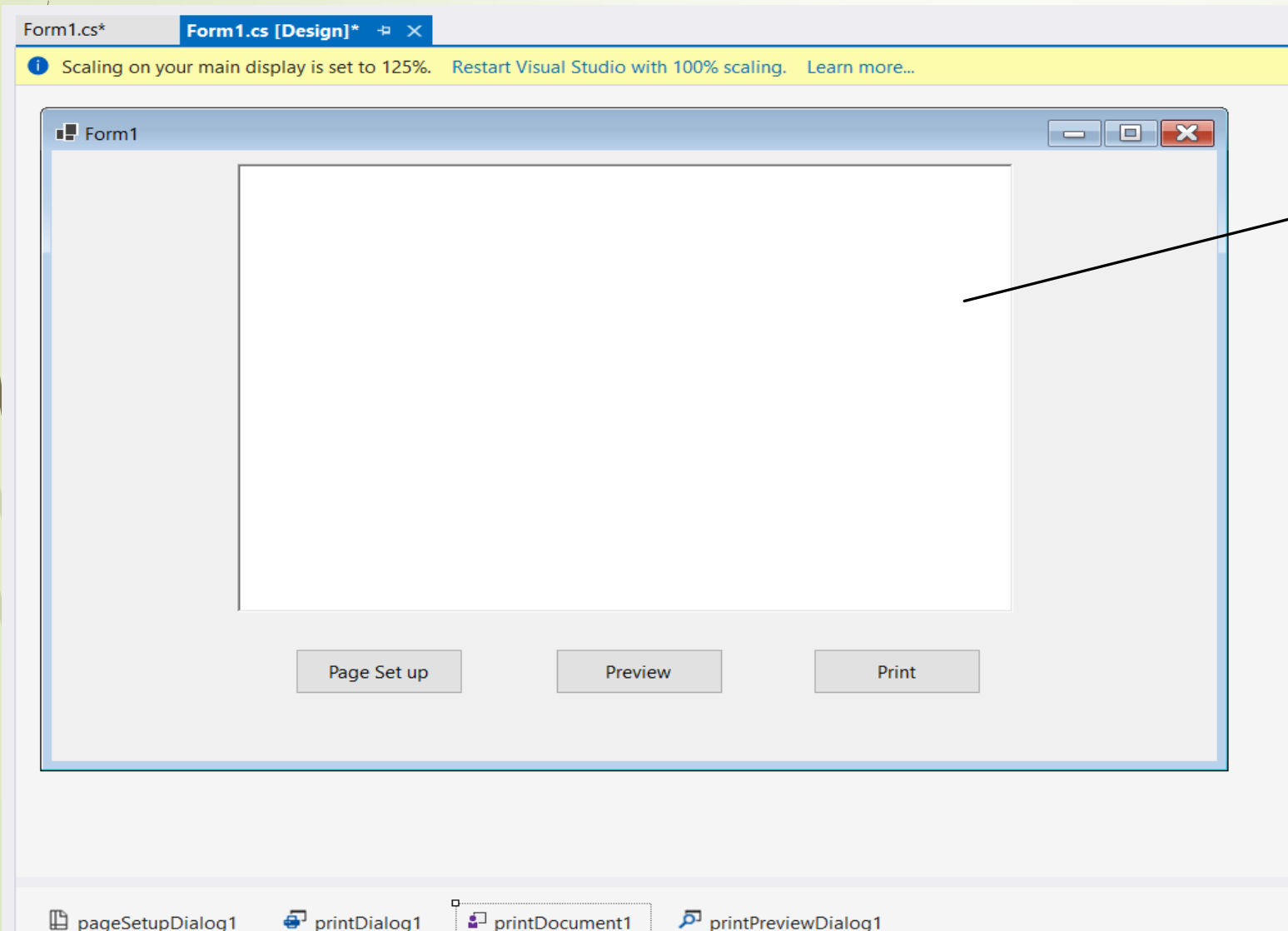
 PrintDialog

 PrintDocument

 PrintPreviewControl

 PrintPreviewDialog

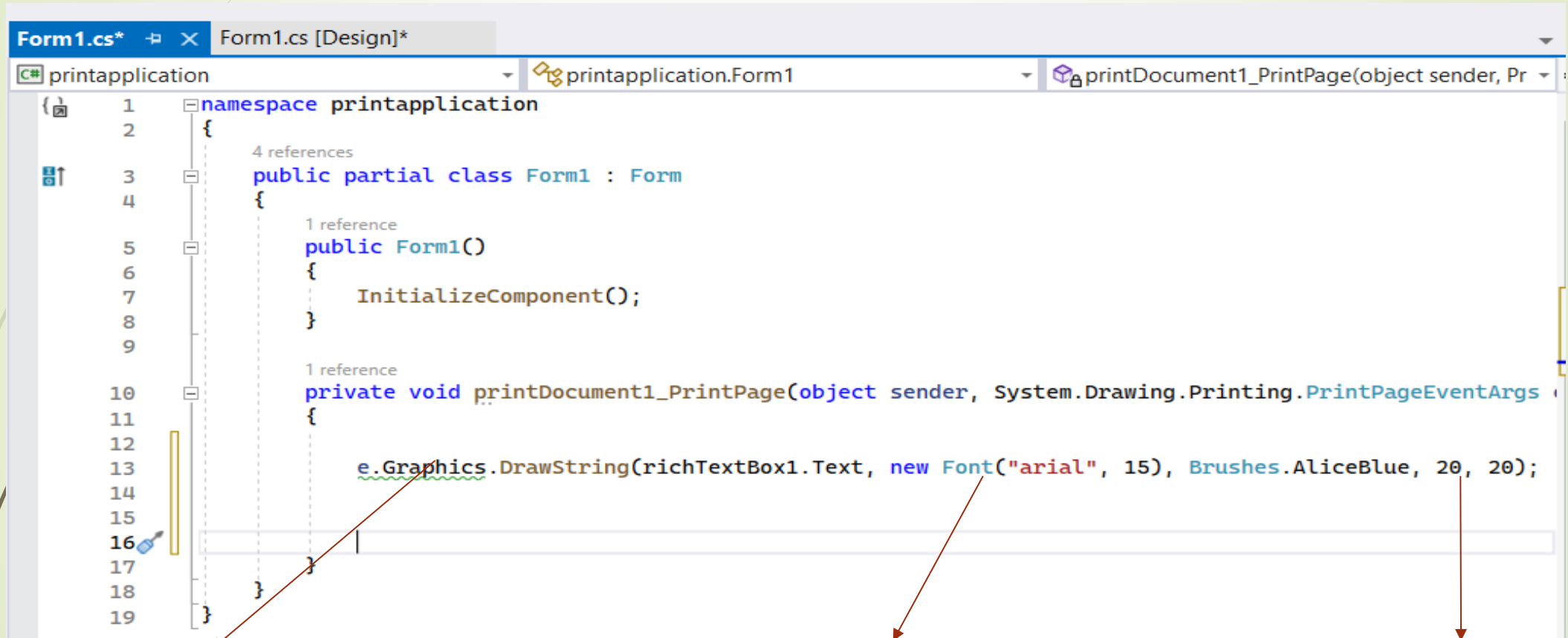
Make the following design then make the document property for the **dialogs** to **printDocument1**



RichTextBox

Behavior	
AllowMargins	True
AllowOrientation	True
AllowPaper	True
AllowPrinter	True
ShowHelp	False
ShowNetwork	True
Data	
Document	printDocument1
MinMargins	0, 0, 0, 0
Tag	

Open the **PrintPage** event and type the following code



```
1 namespace printapplication
2 {
3     4 references
4     public partial class Form1 : Form
5     {
6         1 reference
7         public Form1()
8         {
9             InitializeComponent();
10        }
11
12        1 reference
13        private void printDocument1_PrintPage(object sender, System.Drawing.Printing.PrintPageEventArgs e)
14        {
15            e.Graphics.DrawString(richTextBox1.Text, new Font("arial", 15), Brushes.AliceBlue, 20, 20);
16        }
17    }
18 }
19 }
```

Graphics class is for drawing shapes like arrows, arcs and so on

Print the string or text from the richTextBox1 with specific font and color

Location point(x,y) where the brush draw

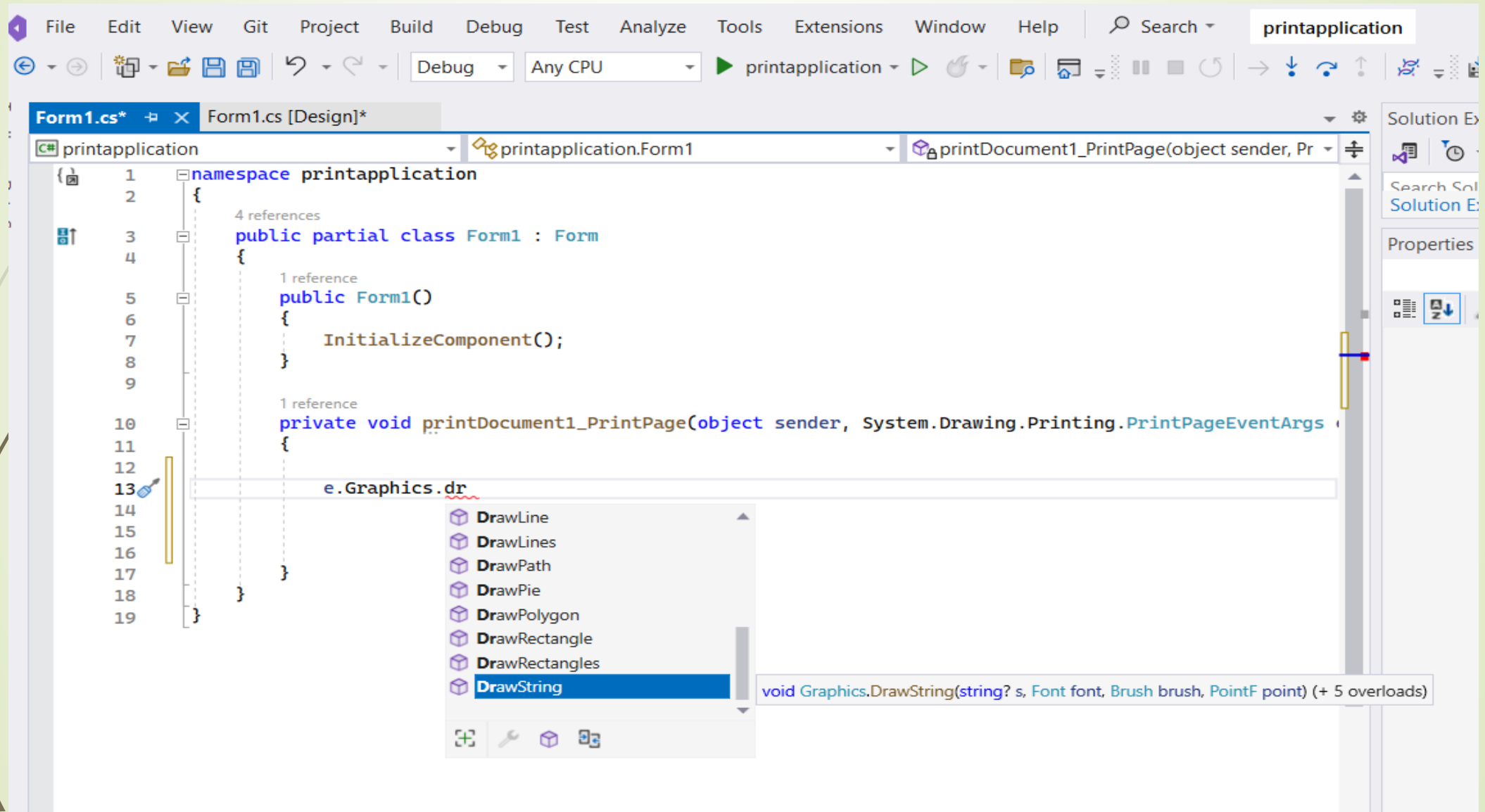
Remember the EventHandler and events

Whenever the mouse is being moved on top of a control, a mouse event is sent. This event is called **MouseMove** and is of type **MouseEventArgs**. It is initiated as follows:

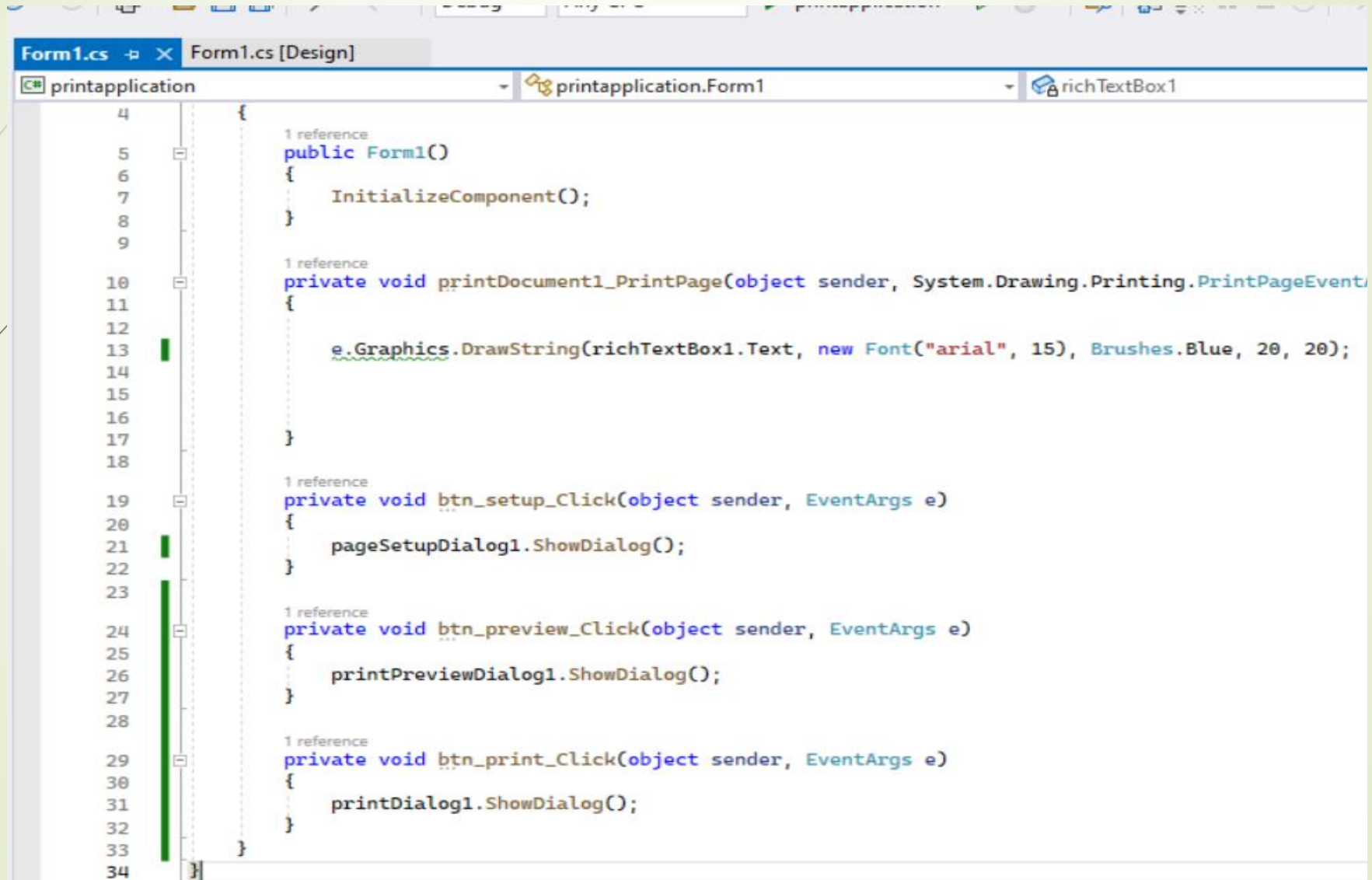
```
private void Control_MouseMove(object sender, System.Windows.Forms.MouseEventArgs e)
{
}
}
```

To implement this event, a **MouseEventArgs** argument is passed to the **EventHandler** event implementer. The **MouseEventArgs** argument provides the necessary information about the event such as what button was clicked, how many times the button was clicked, and the location of the mouse.

DrawString() method print text with specified font type and color this method in Graphics class



Open the Click event for the pagesetup, preview and print buttons



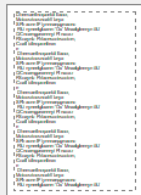
```
4 {
5     1 reference
6     public Form1()
7     {
8         InitializeComponent();
9     }
10
11     1 reference
12     private void printDocument1_PrintPage(object sender, System.Drawing.Printing.PrintPageEvent
13     {
14         e.Graphics.DrawString(richTextBox1.Text, new Font("arial", 15), Brushes.Blue, 20, 20);
15     }
16
17     1 reference
18     private void btn_setup_Click(object sender, EventArgs e)
19     {
20         pageSetupDialog1.ShowDialog();
21     }
22
23     1 reference
24     private void btn_preview_Click(object sender, EventArgs e)
25     {
26         printPreviewDialog1.ShowDialog();
27     }
28
29     1 reference
30     private void btn_print_Click(object sender, EventArgs e)
31     {
32         printDialog1.ShowDialog();
33     }
34 }
```

PrintApplicationForm1

Form1

hello, you are fine ?
here you will take windows programming course

Page Setup



Paper

Size: A4

Source:

Orientation

☒ Portrait

☐ Landscape

Margins (millimetres)

Left: 10 Right: 10

Top: 10 Bottom: 10

OK Cancel

Print preview

       Close

Page 1

hello, you are fine ?
here you will take windows programming course

Page Set up

Preview

Print

Preview

Form1

Print

General

Select Printer

 Fax

 Microsoft Print to PDF

 Microsoft XPS Document Writer

 OneNote (Desktop)

 OneNote for Windows 10

 Send To OneNote 2013

Status: Ready

Location:

Comment:

☐ Print to file

Preferences

Find Printer...

Page Range

☒ All

☐ Selection

☐ Current Page

☐ Pages:

Number of copies: 1

☒ Collate

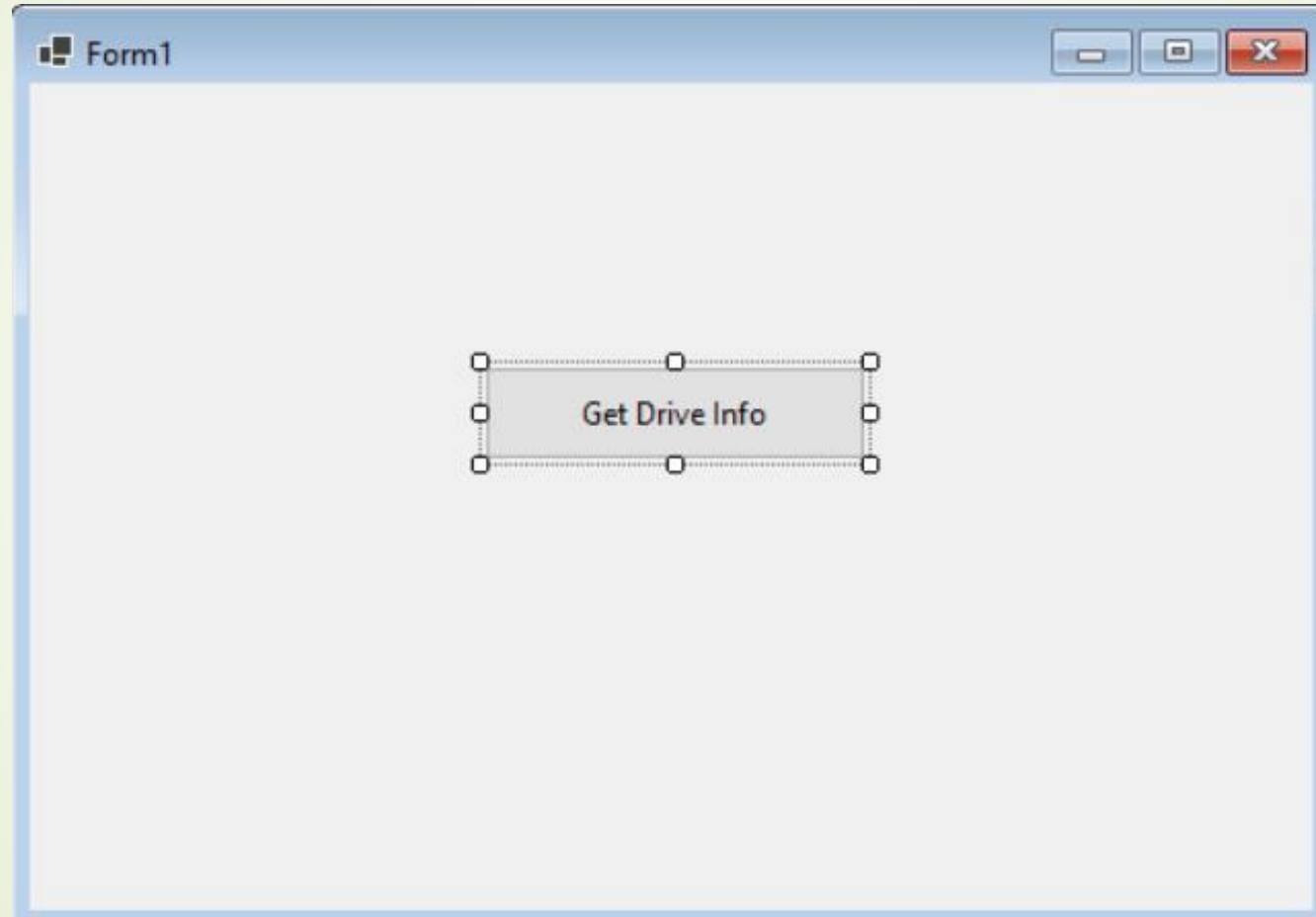
     

Print

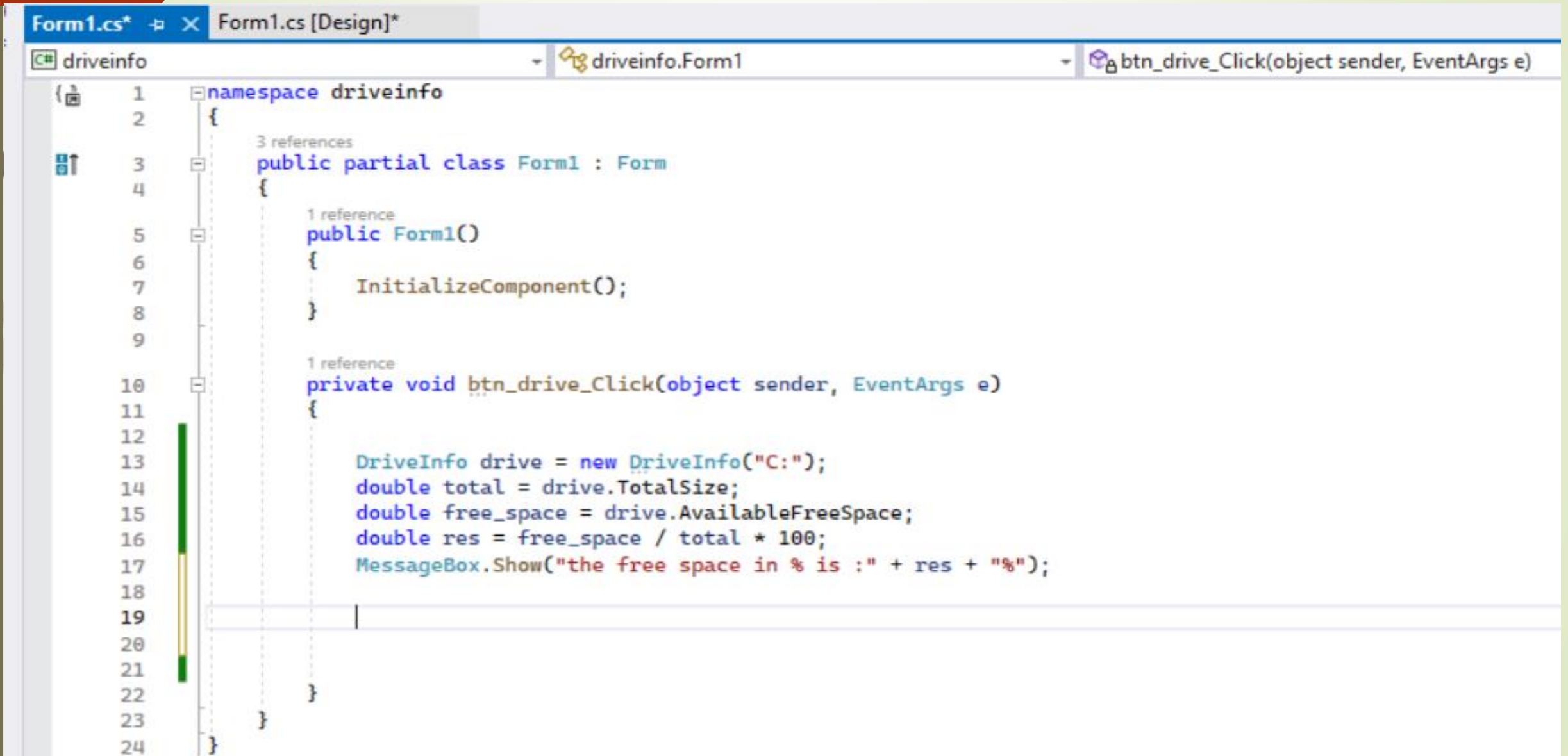
DriveInfo Class

- Computer drive information is a useful feature to have in a program. We can use it to notify the user about insufficient disk space. We get drive information in C# with the **DriveInfo** Class. This class is located inside the System.IO namespace
- A single **DriveInfo** object represents one computer drive. We use the object's properties to get information from that drive. This way we look up its **drive type, total size in bytes, drive name, root directory, and amount of free space.**

Make the following design



In the click event handler put this code



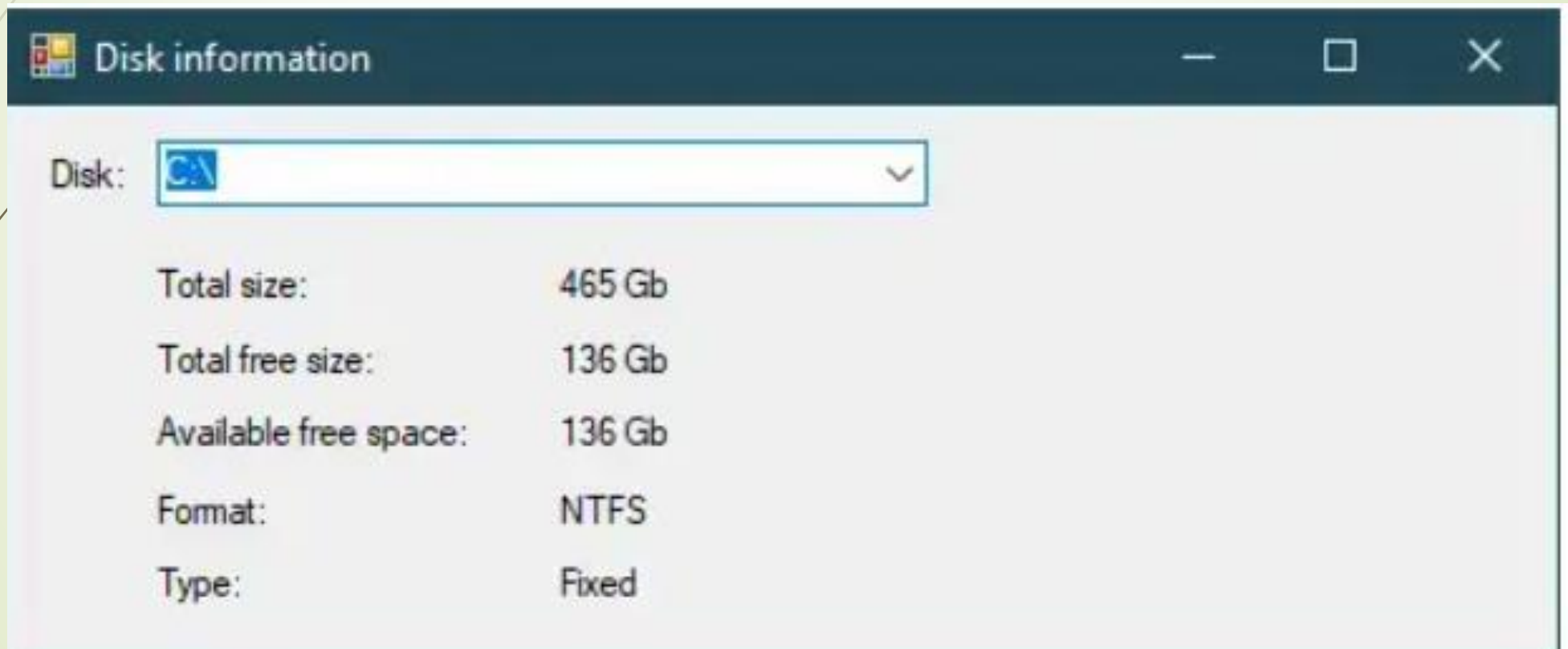
```
Form1.cs*  Form1.cs [Design]*
driveinfo  driveinfo.Form1  btn_drive_Click(object sender, EventArgs e)

1 namespace driveinfo
2 {
3     3 references
4     public partial class Form1 : Form
5     {
6         1 reference
7         public Form1()
8         {
9             InitializeComponent();
10
11         1 reference
12         private void btn_drive_Click(object sender, EventArgs e)
13         {
14             DriveInfo drive = new DriveInfo("C:");
15             double total = drive.TotalSize;
16             double free_space = drive.AvailableFreeSpace;
17             double res = free_space / total * 100;
18             MessageBox.Show("the free space in % is :" + res + "%");
19
20
21
22     }
23 }
24 }
```

Explanation

1. `DriveInfo drive = new DriveInfo("C:");`
 - Accesses information about the C: drive on your computer.
2. `double total = drive.TotalSize;`
 - Gets the total storage of the drive in bytes.
3. `double free_space = drive.AvailableFreeSpace;`
 - Gets the free storage available to the user in bytes.
4. `double res = free_space / total * 100;`
 - Calculates the percentage of free space.
5. `MessageBox.Show("the free space in % is :" + res + "%");`
 - Shows a pop-up message displaying the free space percentage.

Task_1: create the following window and get the disk Information



References

- Developing and Implementing Windows Based Applications with Microsoft Visual C# .NET and Microsoft Visual Studio.
- <https://learn.microsoft.com/en-us/dotnet/api/system.windows.forms.control.focus?view=windowsdesktop-8.0>
- <https://learn.microsoft.com/en-us/dotnet/desktop/winforms/advanced/multiple-document-interface-mdi-applications?view=netframeworkdesktop-4.8>
- <https://kodify.net/csharp/computer-drive/get-drive-info/>